Dexy:
A Simple Stablecoin Design Based on an Algorithmic Central Bank

Alexander Chepurnoy
Ergo Platform
kushti@protonmail.ch

Amitabh Saxena
Ergo Platform

Luca D'Angelo
The Stable Order
ldgaetano@protonmail.com

Abstract

We consider a new algorithmic stablecoin called Dexy consisting of three components: a central bank, a secondary market in the form of a customized CP-AMM liquidity pool, and a trusted oracle. The price of the stablecoin in the liquidity pool is stabilized, relative to the oracle price, via central bank interventions. Users mint Dexy stablecoins from the bank using basecoins, the base cryptocurrency asset.

1 Introduction

Due to the inherent volatility of cryptocurrency prices, algorithmic stablecoins naturally arose as their extension to address this problem. The volatility is resolved by pegging the stablecoin to an asset whose price is considered to be stable over long periods of time, e.g. gold.

Having an asset with a stable value can be useful in many scenarios:

- Securing fundraising: a project can be sure that funds collected during fundraising will have stable value in the mid-to-long term.

We would would like to thank Ile for his inspiring forum posts, Bruno for discussions and the extract to future idea, and 4EYES Labs for the audit of the ErgoScript contracts.

- Doing business with predictable results: a shop can be sure that funds collected from sales will have roughly the same value when the shop is ordering goods from warehouses, otherwise, the shop may go bankrupt if its margins are to low to cover exchange rate fluctuations.

- Shorting: when cryptocurrency prices are high, it is desirable for investors to rebalance their portfolio by increasing exposure to fiat currencies or real-world commodities. However, as fiat currencies and centralized exchanges impose significant risks, it would be better to buy fiat and commodity substitues in the form of stablecoins on decentralized exchanges.

- Lending and other decentralized finance applications: stability of collateral value is critical for many applications built on top of these services.

Centralized stablecoins, such as USDT and USDC, use a trusted party to ensure that the peg is maintained. For algorithmic stablecoins, the peg can be maintained in different ways depending on the implementation, e.g. rebasement of the total supply [4], or via imitating a trusted party that holds reserves and then does market interventions for rebalancing the peg. Imititating the trusted party is usually done by allowing anyone on the blockchain to create over-collateralized financial instruments, such as collateralized debt positions [3], zero-coupon bonds (as was done in the failed Yield protocol), reserve assets [11, 12, 8], or by issuing stabilizing financial instruments in case of a depeg [5].

In this work, we present Dexy, a stablecoin protocol where an algorithmic central bank performs interventions when there is a depeg. The bank stabilizes the stablecoin value in the reference market, a liquidity pool, by injecting basecoins from its reserves when the stablecoin price is below the peg. In extreme cases, when bank reserves are depleted and the stablecoin price is still below the peg, the liquidity pool price is restored by extracting stablecoins from it, these will be injected back again at a future time, when the stablecoin price is above the peg by a certain threshold. In some aspects, Dexy could resemble Fei or Gyroscope [7].

The rest of the paper is organized as follows. In Section 2, the Dexy design in general is explained, where important trust assumptions are stated and notation used in other sections is introduced. Section 4 defines the worst-case scenario for the bank reserves, and so the stablecoin stability as well. In Section 5, the bank and liquidity pool rules are defined, where their stability is discussed in Section 6. Section 8 describes Dexy implementations: the gold-pegged DexyGold stablecoin and the USD-pegged USE stablecoin.

2 Dexy Design

The Dexy protocol functions via an algorithmic central bank that holds reserves in basecoins and mints stablecoins. A corresponding liquidity pool exists where stablecoins and basecoins can be traded. The peg price used by the bank is provided by an oracle, which is assumed to be trusted and reliable. Users can interact with both the bank and the liqudity pool, but certain rules and restrictions are applied to control the price stability. Central bank interventions are determined by the ratio of the liquidity pool price to the oracle price, $\bar{r}$, which we also call the reference reserve ratio.

2.1 The Liquidity Pool

The reference market is implemented as an automated market maker (e.g. [10]). For simplicity, a modified constant-product automated market maker (CP-AMM) is used, similar to ErgoDex [1] and UniSwap [6]. This means that the liqudity pool holds a pair of basecoins and stablecoins such that the product of their amounts remains invariant before and after a trade. Users who function as liquidity providers can provide a pair of basecoins and stablecoins to obtain a corresponding amount of LP tokens. These LP tokens can later be deposited to withdraw their initial liquidity pair deposit plus accumulated fees from trading. A fee is charged to liqudity providers to prevent excessive liquidity withdrawals.

2.2 The Algorithmic Central Bank

By depositing basecoins into the bank, users can mint stablecoins. The price provided by the bank will always equal the oracle price. A fee is charged so the bank reserves can increase faster, along with a fee for the oracle usage. Under certain conditions, the bank can intervene into the liquidity pool to control its price relative to the oracle price.

2.3 Trust Assumptions

In general, for any DeFi protocol involving user funds, it is important for trust assumptions to be explicitly stated so that users can make informed decisions to the best of their ability.

The Dexy protocol has two:

- Oracle: The protocol assumes the target price oracle is reliable. This trust assumption is unavoidable, however, it can be relaxed by using a federation of oracles that produces an average price to be delivered to the central bank, instead of relying on a single entity. In any case, it is important that oracles are rewarded from protocol usage, otherwise, they will be incentivized to adversarially attack the protocol, see Section **??** for details.

- No governance: Unlike most stablecoin protocols, we do not consider governance, as it can be used to adversarially attack the protocol. We suppose that real-world instantiations that do have it should clearly state their governance-related trust assumptions.

2.4 Actions and Dynamics

The following table summarizes the different actions in the protocol, the $\bar{r}$ range when the action happens, and the restored $\bar{r}'$ range after the action happens.

Action	Range	Restore Range
FreeMint	$1 - \epsilon_{\text{FREE}} \leq \bar{r} \leq 1 + \epsilon_{\text{FREE}}$	-
ArbitrageMint	$\bar{r} > 1 + \epsilon_{\text{FREE}}$	-
InjectBasecoins	$\bar{r} < 1 - \epsilon_{\text{FREE}}$	$1 - \epsilon_{\text{FREE}} \leq \bar{r}' \leq 1 - \epsilon_{\text{IB}}$
ExtractDexy	$\bar{r} \leq 1 - \epsilon^0_{\text{ED}} < 1 - \epsilon_{\text{FREE}}$	$1 - \epsilon_{\text{FREE}} - \epsilon^1_{\text{ED}} \leq \bar{r}' \leq 1 - \epsilon_{\text{FREE}}$
ReleaseDexy	$1 + \epsilon_{\text{RD}} < \bar{r} < 1 + \epsilon_{\text{FREE}}$	$\bar{r}' \geq 1 + \epsilon_{\text{RD}}$
StoppedWithdrawals	$\bar{r} < 1 - \epsilon_{\text{FREE}}$	-

We now give a high-level description of what happens during each action. For more technical details, refer to section 3.

2.4.1 FreeMint

- $\bar{r}$ is bounded within the range $1 - \epsilon_{\text{FREE}} \leq \bar{r} \leq 1 + \epsilon_{\text{FREE}}$.
- If bounded within the range, then minting can occur for a fixed period of time T_{FREE}.
- Users can mint from the bank up to a maximum amount determined by the Dexy volume in the liquidity pool at the beginning
- The bank charges a fee so the bank reserves can increase faster.
- **Consequence:** Increase bank reserves and stablecoin supply when the pool price is considered stable.

2.4.2 ArbitrageMint

- $\bar{r}$ is greater than $1 + \epsilon_{\text{FREE}}$.
- The $\bar{r}$ is above the threshold for an amount of time greater than τ_{ARB}.
- The liquidity pool and oracle price difference determines the max amount of stablecoins the bank can mint and the corresponding max amount of basecoins it will accept.

- The bank charges a fee so the bank reserves can increase faster.
- **Consequence:** Purchase stablecoins from the bank at a discount and sell them to the liquidity pool at premium, increasing the liquidity pool stablecoin supply.

2.4.3 InjectBasecoins

- $\bar{r}$ is less than $1 - \epsilon_{\text{FREE}}$.
- The $\bar{r}$ is less than the threshold for an amount of time greater than τ_{IB}.
- The bank still has basecoin reserves.
- The bank will inject basecoins from its reserves into the liquidity pool until the new $\bar{r}$ is within the range $1 - \epsilon_{\text{FREE}} \leq \bar{r}' \leq 1 - \epsilon_{\text{IB}}$.
- The amount of basecoins injected is a fixed percentage α_{FREE} of the banks total basecoin reserve.
- **Consequence:** The bank's basecoins reserves decrease and the liquidity pool price recovers.

2.4.4 ExtractDexy

- $\bar{r}$ is less than $1 - \epsilon^0_{\text{ED}} < 1 - \epsilon_{\text{FREE}}$.
- $\bar{r}$ is less than the threshold for an amount of time greater than $\tau_{\text{ED}} << \tau_{\text{IB}}$
- The bank has no more basecoin reserves.
- The bank will extract stablecoins from the liquidity pool until the new $\bar{r}$ is in the range $1 - \epsilon_{\text{FREE}} - \epsilon^1_{\text{ED}} \leq \bar{r}' \leq 1 - \epsilon_{\text{FREE}}$, so essentially the stopped-withdrawl state, and then lock them to be released at a future time.
- **Consequence:** The bank removes stablecoins from the liquidity pool and the liquidity pool price recovers.

2.4.5 ReleaseDexy

- $\bar{r}$ is within the range $1 + \epsilon_{\text{RD}} < \bar{r} < 1 + \epsilon_{\text{FREE}}$.
- $\bar{r}$ is within the range for an amount of time greater than τ_{RD}.

- The bank will release the locked stablecoins back into the liquidity pool until the new $\bar{r}$ is greater than or equal to $1 + \epsilon_{\text{RD}}$
- **Consequence:** The bank releases stablecoins back into the liquidity pool such that it is prioritized over arbitrage minting.

2.4.6 StoppedWithdrawals

- $\bar{r}$ is less than $1 - \epsilon_{\text{FREE}}$.
- Liquidity pair withdrawals are stopped without delay once the $\bar{r}$ intervention threshold is met.
- Only swaps between basecoins and stablecoins is allowed.
- **Consequence:** Liquidity providers, i.e. LP token holders, cannot withdraw their liquidity pair from the liquidity pool to prevent further price depegging.

3 Protocol Specification

We now describe the Dexy protocol specification, which includes immutable protocol paramters and state variables. We note the distinction between an action period, which measures how long the bank is in a particular state, and a corresponding action bank delay, which measures how long before a bank action can be performed, i.e. the delay before the start of the first bank action period. For a particular bank action, there may be multiple periods before a new delay is required. Whether or not these are set to be equal is an implementation decision.

The central bank parameters are a tuple C_{BANK} of:

$$\left(\phi^{\text{FREE}}_{\text{BANK}}, \phi^{\text{ARB}}_{\text{BANK}}, \epsilon_{\text{FREE}}, \epsilon_{\text{IB}}, \epsilon^{0}_{\text{ED}}, \epsilon^{1}_{\text{ED}}, \epsilon_{\text{RD}}, T_{\text{FREE}}, T_{\text{ARB}}, T_{\text{IB}}, \tau_{\text{IB}}, \tau_{\text{ED}}, \tau_{\text{RD}}, \tau_{\text{TX}}, \alpha_{\text{FREE}}, \alpha_{\text{IB}}\right)$$

- $\phi^{\text{FREE}}_{\text{BANK}} \in (0, 1)$ is the bank fee for the free-mint action.
- $\phi^{\text{ARB}}_{\text{BANK}} \in (0, 1)$ is the bank fee for the arbitrage-mint action.
- $\epsilon_{\text{FREE}} \in (0, 1)$ defines the upper and lower bounds of the free-mint range, representing the percentage that the LP price deviates from the oracle price.
- $0 < \epsilon_{\text{IB}} < \epsilon_{\text{FREE}}$ defines the percentage where the inject-basecoins action will restore the price.
- $\epsilon_{\text{FREE}} < \epsilon^{0}_{\text{ED}} < 1$ defines the percentage where the extract-dexy action will happen.

- $0 < \epsilon^1_{\text{ED}} < \epsilon^0_{\text{ED}}$ defines the percentage where the extract-dexy action will restore the price.
- $\epsilon_{\text{RD}} < \epsilon_{\text{FREE}}$ defines the percentage where the release-dexy action will happen.
- $T_{\text{FREE}} \in \mathbb{R}_{>0}$ is the free-mint period.
- $T_{\text{ARB}} \in \mathbb{R}_{>0}$ is the arbitrage-mint period.
- $T_{\text{IB}} \in \mathbb{R}_{>0}$ is the inject-basecoins period.
- $\tau_{\text{ARB}} \in \mathbb{R}_{>0}$ is the arbitrage-mint bank delay.
- $\tau_{\text{IB}} \in \mathbb{R}_{>0}$ is the inject-basecoins bank delay.
- $\tau_{\text{ED}} >> \tau_{\text{IB}}$ is the extract-dexy bank delay, which must be much larger than the inject-basecoins bank delay, since extraction is an action of last-resort.
- $\tau_{\text{RD}} \in \mathbb{R}_{>0}$ is the release-dexy bank delay.
- $\tau_{\text{TX}} \in \mathbb{R}_{>0}$ is the allowed transaction propogation delay.
- $\alpha_{\text{FREE}} \in (0, 1)$ is the percentage of dexy liquidity, in the liquidity pool, that determines the max amount of dexy that can be minted from the bank during the free-mint period.
- $\alpha_{\text{IB}} \in (0, 1)$ is the percentage of the bank's basecoin reserve that determines the max amount of basecoins to inject into the liquidity pool during an inject-basecoin action.

The central bank state is determined by a tuple of two values:

$$s_{\text{BANK}} = (R, S_D)$$

- $R \in \mathbb{R}_{>0}$ is the bank's basecoins reserves.
- $S_D \in \mathbb{R}_{>0}$ is the total supply of minted dexy stablecoins.

The liquidity pool has two parameter:

$$C_{LP} = \left(\phi^{REDEEM}_{LP}, \phi^{SWAP}_{LP}\right)$$

- $\phi^{REDEEM}_{LP} \in (0, 1)$ is the liquidity provider withdrawal fee, which should be high enough to disincentivize frequent withdrawals.
- $\phi^{SWAP}_{LP} \in (0, 1)$ is the liquidity pool swap fee, which is used to reward liquidity providers.

The liquidity pool state is determined by a tuple of three values:

$$s_{\text{LP}} = (B, D, S_{LP})$$

- $B \in \mathbb{R}_{>0}$ is the amount of basecoins in the liquidity pool.
- $D \in \mathbb{R}_{>0}$ is the amount of dexy stablecoins in the liquidity pool.
- $S_{LP} \in \mathbb{R}_{>0}$ is the supply of LP tokens provided to liquidity providers.

The oracle parameters are assumed to only be the following:

$$C_O = (\phi_O)$$

- $\phi_O \in (0, 1)$ is the oracle fee.

The oracle state is determined by a tuple of two values:

$$s_{\text{O}} = (P^*_{\text{D}}, F)$$

- $P^*_{\text{D}} \in \mathbb{R}_{>0}$ is the target stablecoin price, in units of $\frac{\text{BASECOIN}}{\text{PEG ASSET}}$.
- $F \in \mathbb{R}_{\geq 0}$ is the accumulated fees of the oracle.

For the sake of clarity, we now define separate state variables for the different bank actions, but these can also be considered as part of the bank state.

The FreeMint action has the following state:

$$s_{\text{FREE}} = (t, d)$$

- $t \in \mathbb{R}_{>0}$ is the time when the free-mint period ends.
- $d \in \mathbb{R}_{\geq 0}$ is the remaining amount of dexy stablecoins that can be minted during the current free-mint period, determined by t.

The ArbMint action has the following state:

$$s_{\text{ARB}} = (t, d, \tau)$$

- $t \in \mathbb{R}_{>0}$ is the time when the arbitrage-mint period ends.
- $d \in \mathbb{R}_{\geq 0}$ is the remaining amount of dexy stablecoins that can be minted during the current arbitrage-mint period, determined by t.

- $\tau \in \mathbb{R}_{>0} \cup \{\infty\}$ is the time when $r > 1 + \epsilon_{\text{FREE}}$. This can occur more than once as time goes on, each occurrence corresponding to a potentially new arbitrage-mint period.

The InjectBasecoin action has the following state:

$$s_{\text{IB}} = (t, \tau)$$

- $t \in \mathbb{R}_{>0}$ is the time when the previous inject-basecoin action occurred.
- $\tau \in \mathbb{R}_{>0} \cup \{\infty\}$ is the time when $r < 1 + \epsilon_{\text{FREE}}$. This can occur more than once as time goes on, each occurrence corresponding to a potentially new inject-basecoin period.

The ExtractDexy and ReleaseDexy actions share the following state:

$$s_{\text{BURN}} = \left(t_{\text{ED}}, t_{\text{RD}}, d, \tau^{\text{ED}}, \tau^{\text{RD}}\right)$$

- $t_{\text{ED}} \in \mathbb{R}_{>0}$ is the time when the previous extract-dexy action occurred.
- $t_{\text{RD}} \in \mathbb{R}_{>0}$ is the time when the previous release-dexy action occurred.
- $d \in \mathbb{R}_{\geq 0}$ is the remaining dexy amount to be released.
- $\tau^{\text{ED}} \in \mathbb{R}_{>0} \cup \{\infty\}$ is the time when $r \leq 1 - \epsilon^0_{\text{ED}}$.
- $\tau^{\text{RD}} \in \mathbb{R}_{>0} \cup \{\infty\}$ is the time when $r > 1 + \epsilon_{\text{RD}}$.

3.1 Auxiliary Definitions

Definition 1 (Bank Liabilities)**.** *The bank liabilities is defined as the product of the oracle price and the minted supply of dexy stablecoins:*

$$L(P^*_{\text{D}}, s_{BANK}) = P^*_{\text{D}} S_{\text{D}} \tag{1}$$

Definition 2 (Bank Reserve Ratio)**.** *The bank reserve ratio is defined as the ratio of its reserves to its liabilities:*

$$r(P^*_{\text{D}}, s_{BANK}) = \frac{R}{L(P^*_{\text{D}}, s_{BANK})} = \frac{R}{P^*_{\text{D}} S_{\text{D}}} \tag{2}$$

Assuming the oracle price is constant, and the reserves are greater than the liabilities, the bank reserve ratio will always increase after stablecoins are minted. During periods of sudden basecoin price crashes, the reserve ratio will decrease. Since users cannot sell dexy stablecoins to the bank, the bank is liable to no one, thus the bank reserves can be less than its liabilities, which is expected during bank interventions. So, a measure of the health of the $Dexy$ protocol is having an increasing reserve ratio, before and after bank interventions.

Definition 3 (Reference Price)**.** *The liquidity pool price, or the reference market price, is defined as follows:*

$$P_{\mathrm{D}}^{\mathrm{LP}}(s_{\mathrm{LP}}) = \frac{B}{D} \tag{3}$$

The reference price, $P_{\mathrm{D}}^{\mathrm{LP}}$*, has units of* $\frac{\textit{BASECOIN}}{\textit{STABLECOIN}}$*.*

Definition 4 (Reference Reserve Ratio)**.** *The ratio of the liquidity pool price to the oracle price is called the reference reserve ratio:*

$$\bar{r}(P_{\mathrm{D}}^{*}, s_{\mathrm{LP}}) = \frac{P_{\mathrm{D}}^{\mathrm{LP}}(s_{\mathrm{LP}})}{P_{\mathrm{D}}^{*}} = \frac{B}{P_{\mathrm{D}}^{*} D} \tag{4}$$

The aim of the $Dexy$ *protocol is to maintain* $\bar{r}$ *within a sufficiently small neighbourhood of 1.* $\bar{r}$ *is called the reference reserve ratio since we can interpret the formula as the reserve ratio of the liquidity pool.*

3.2 State Update Procedures

In order to perform each bank action, we must know both the current value of $\bar{r}$ but also when the transition from one region to another occurred. The following state update procedures ensure that the bank always has the accurate state information available before any bank action is performed. The procedures are meant to be continuously running, checking which bank action states can be updated according to whatever value the $\bar{r}$ has.

In practice, on eUTxO blockchains, these procedures are validators, such that if they evaluate to TRUE, then the corresponding transaction built to update the state can be accepted by a node and successfully submitted to the blockchain for inclusion.

3.2.1 UpdateArbitrageMint

Algorithm 1 UpdateArbitrageMint

Require: $s_{\text{O}}, s_{\text{LP}}, s_{\text{ARB}}$, Update time t', Current time t

$(\tilde{t}, \tilde{d}, \tilde{\tau}) \leftarrow s_{\text{ARB}}$
$(P^*_{\text{D}}, F) \leftarrow s_{\text{O}}$
$\bar{r} \leftarrow \bar{r}(P^*_{\text{D}}, s_{\text{LP}})$
$notUpdated \leftarrow (\tau == \infty)$
$validUpdate \leftarrow (t' \geq t - \tau_{\text{TX}}) \wedge (t' \leq t)$ ▷ Update time is in a valid interval.
$notReset \leftarrow (\tau < \infty)$
$validReset \leftarrow (t' == \infty)$
$isUpdate \leftarrow (\bar{r} > 1 + \epsilon_{\text{FREE}}) \wedge notUpdated \wedge validUpdate$
$isReset \leftarrow (\bar{r} \leq 1 + \epsilon_{\text{FREE}}) \wedge notReset \wedge validReset$
if $isUpdate \vee isReset$ **then**
    $s_{\text{ARB}} \leftarrow (\tilde{t}, \tilde{d}, t')$
    return TRUE
else
    return FALSE
end if

3.2.2 UpdateInjectBasecoins

Algorithm 2 UpdateInjectBasecoins

Require: $s_{\mathrm{O}}, s_{\mathrm{LP}}, s_{\mathrm{IB}}$, Update time t', Current time t

$(\tilde{t}, \tilde{\tau}) \leftarrow s_{\mathrm{IB}}$
$(P^*_{\mathrm{D}}, F) \leftarrow s_{\mathrm{O}}$
$\bar{r} \leftarrow \bar{r}(P^*_{\mathrm{D}}, s_{\mathrm{LP}})$
$notUpdated \leftarrow (\tau == \infty)$
$validUpdate \leftarrow (t' \geq t - \tau_{\mathrm{TX}}) \wedge (t' \leq t)$ ▷ Update time is in a valid interval.
$notReset \leftarrow (\tau < \infty)$
$validReset \leftarrow (t' == \infty)$
$isUpdate \leftarrow (\bar{r} < 1 - \epsilon_{\mathrm{FREE}}) \wedge notUpdated \wedge validUpdate$
$isReset \leftarrow (\bar{r} \geq 1 - \epsilon_{\mathrm{FREE}}) \wedge notReset \wedge validReset$
if $isUpdate \vee isReset$ **then**
    $s_{\mathrm{IB}} \leftarrow (\tilde{t}, t')$
    return TRUE
else
    return FALSE
end if

3.2.3 UpdateExtractDexy

Algorithm 3 UpdateExtractDexy

Require: $s_{\text{O}}, s_{\text{LP}}, s_{\text{BURN}}$, Update time t', Current time t

$(\tilde{t}_{\text{ED}}, \tilde{t}_{\text{RD}}, \tilde{d}, \tilde{\tau}^{\text{ED}}, \tilde{\tau}^{\text{RD}}) \leftarrow s_{\text{BURN}}$
$(P^*_{\text{D}}, F) \leftarrow s_{\text{O}}$
$\bar{r} \leftarrow \bar{r}(P^*_{\text{D}}, s_{\text{LP}})$
$notUpdated \leftarrow (\tau == \infty)$
$validUpdate \leftarrow (t' \geq t - \tau_{\text{TX}}) \wedge (t' \leq t)$ ▷ Update time is in a valid interval.
$notReset \leftarrow (\tau < \infty)$
$validReset \leftarrow (t' == \infty)$
$isUpdate \leftarrow (\bar{r} \leq 1 - \epsilon^0_{\text{ED}}) \wedge notUpdated \wedge validUpdate$
$isReset \leftarrow (\bar{r} > 1 - \epsilon^0_{\text{ED}}) \wedge notReset \wedge validReset$
if $isUpdate \vee isReset$ **then**
    $s_{\text{BURN}} \leftarrow (\tilde{t}_{\text{ED}}, \tilde{t}_{\text{RD}}, \tilde{d}, t', \tilde{\tau}^{\text{RD}})$
    return TRUE
else
    return FALSE
end if

3.2.4 UpdateReleaseDexy

Algorithm 4 UpdateReleaseDexy

Require: $s_{\text{O}}, s_{\text{LP}}, s_{\text{BURN}}$, Update time t', Current time t

$(\tilde{t}_{\text{ED}}, \tilde{t}_{\text{RD}}, \tilde{d}, \tilde{\tau}^{\text{ED}}, \tilde{\tau}^{\text{RD}}) \leftarrow s_{\text{BURN}}$
$(P^*_{\text{D}}, F) \leftarrow s_{\text{O}}$
$\bar{r} \leftarrow \bar{r}(P^*_{\text{D}}, s_{\text{LP}})$
$notUpdated \leftarrow (\tau == \infty)$
$validUpdate \leftarrow (t' \geq t - \tau_{\text{TX}}) \wedge (t' \leq t)$ ▷ Update time is in a valid interval.
$notReset \leftarrow (\tau < \infty)$
$validReset \leftarrow (t' == \infty)$
$isUpdate \leftarrow (\bar{r} > 1 + \epsilon_{\text{RD}}) \wedge notUpdated \wedge validUpdate$
$isReset \leftarrow (\bar{r} \leq 1 + \epsilon_{\text{RD}}) \wedge notReset \wedge validReset$
if $isUpdate \vee isReset$ **then**
    $s_{\text{BURN}} \leftarrow (\tilde{t}_{\text{ED}}, \tilde{t}_{\text{RD}}, \tilde{d}, \tilde{\tau}^{\text{ED}}, t')$
    return TRUE
else
    return FALSE
end if

3.3 Protocol Operations

3.3.1 FreeMint

Algorithm 5 FreeMint

Require: $s_{\text{O}}, s_{\text{LP}}, s_{\text{BANK}}, s_{\text{FREE}}$, Basecoins b, Updated period end time T', Current time t

$(\tilde{t}, \tilde{d}) \leftarrow s_{\text{FREE}}$
$(R, S_{\text{D}}) \leftarrow s_{\text{BANK}}$
$(B, D) \leftarrow s_{\text{LP}}$
$(P^*_{\text{D}}, F) \leftarrow s_{\text{O}}$
$\bar{r} \leftarrow \bar{r}(P^*_{\text{D}}, s_{\text{LP}})$
$\tilde{P}^*_{\text{D}} \leftarrow P^*_{\text{D}}(1 + \phi^{\text{FREE}}_{\text{BANK}} + \phi_O)$
$isCounterReset \leftarrow (t > \tilde{t})$ ▷ If true, then the protocol is in a new FreeMint period.
$dexyMinted \leftarrow \frac{b}{\tilde{P}^*_{\text{D}}}$
$oracleFeeAmount \leftarrow dexyMinted * P^*_{\text{D}} * \phi_O$
$\delta_d \leftarrow \alpha_{\text{FREE}} * D$ ▷ Represents the maximum bank reserve growth during the FreeMint period.
$dexyAvailable \leftarrow isCounterReset \ ? \ \delta_d : \tilde{d}$
$validAmount \leftarrow dexyMinted \leq dexyAvailable$
if $\neg isCounterReset$ **then**
    $T' \leftarrow \tilde{t}$
    $validUpdate \leftarrow$ TRUE
else
    $validUpdate \leftarrow T' \in [t + T_{\text{FREE}}, t + T_{\text{FREE}} + \tau_{\text{TX}}]$ ▷ Checks new period end time.
end if
$validRatio \leftarrow \bar{r} \in (1 - \epsilon_{\text{FREE}}, 1 + \epsilon_{\text{FREE}})$
if $validUpdate \wedge validRatio$ **then**
    $s_{\text{BANK}} \leftarrow (R + b, S_{\text{D}} + dexyMinted)$
    $s_{\text{O}} \leftarrow (P^*_{\text{D}}, F + oracleFeeAmount)$
    $s_{\text{FREE}} \leftarrow (T', dexyAvailable - dexyMinted)$
    return TRUE
else
    return FALSE
end if

3.3.2 ArbitrageMint

Algorithm 6 ArbMint

Require: $s_{\text{O}}, s_{\text{LP}}, s_{\text{BANK}}, s_{\text{ARB}}$, Basecoins b, Updated period end time T', Current time t

$(\tilde{t}, \tilde{d}, \tilde{\tau}) \leftarrow s_{\text{ARB}}$
$(R, S_{\text{D}}) \leftarrow s_{\text{BANK}}$
$(B, D) \leftarrow s_{\text{LP}}$
$(P^*_{\text{D}}, F) \leftarrow s_{\text{O}}$
$\bar{r} \leftarrow \bar{r}(P^*_{\text{D}}, s_{\text{LP}})$
$\tilde{P}^*_{\text{D}} \leftarrow P^*_{\text{D}}(1 + \phi^{\text{FREE}}_{\text{BANK}} + \phi_O)$
$isCounterReset \leftarrow (t > \tilde{t})$ ▷ If true, then the protocol is in a new ArbMint period.
$dexyMinted \leftarrow \frac{b}{\tilde{P}^*_{\text{D}}}$
$oracleFeeAmount \leftarrow dexyMinted * P^*_{\text{D}} * \phi_O$
$\delta_b \leftarrow B - D * \tilde{P}^*_{\text{D}}$
$\delta_d \leftarrow \frac{\delta_b}{\tilde{P}^*_{\text{D}}}$ ▷ Represents the maximum bank reserve growth during the ArbMint period.
$dexyAvailable \leftarrow isCounterReset \ ? \ \delta_d : \tilde{d}$
$validAmount \leftarrow dexyMinted \leq dexyAvailable$
if $\neg isCounterReset$ **then**
    $T' \leftarrow \tilde{t}$
    $validUpdate \leftarrow$ TRUE
else
    $validUpdate \leftarrow T' \in [t + T_{\text{ARB}}, t + T_{\text{ARB}} + \tau_{\text{TX}}]$ ▷ Checks new period end time.
end if
$validDelay \leftarrow t > \tilde{\tau} + \tau_{\text{ARB}}$
$validRatio \leftarrow \bar{r} > 1 + \epsilon_{\text{FREE}}$
if $validUpdate \wedge validDelay \wedge validRatio$ **then**
    $s_{\text{BANK}} \leftarrow (R + b, S_{\text{D}} + dexyMinted)$
    $s_{\text{O}} \leftarrow (P^*_{\text{D}}, F + oracleFeeAmount)$
    $s_{\text{ARB}} \leftarrow (T', dexyAvailable - dexyMinted, \tilde{\tau})$
    return TRUE
else
    return FALSE
end if

3.3.3 InjectBasecoins

Algorithm 7 InjectBasecoins

Require: $s_{\text{O}}, s_{\text{LP}}, s_{\text{BANK}}, s_{\text{IB}}$, Basecoins b, Tx creation time t', Current time t

$(\tilde{t}, \tilde{\tau}) \leftarrow s_{\text{IB}}$
$(R, S_{\text{D}}) \leftarrow s_{\text{BANK}}$
$(B, D) \leftarrow s_{\text{LP}}$
$(P^*_{\text{D}}, F) \leftarrow s_{\text{O}}$
$s'_{\text{LP}} \leftarrow (B + b, D)$
$\bar{r} \leftarrow \bar{r}(P^*_{\text{D}}, s_{\text{LP}})$
$\bar{r}' \leftarrow \bar{r}(P^*_{\text{D}}, s'_{\text{LP}})$
$validAmount \leftarrow b \leq \alpha_{\text{IB}} * S_{\text{D}}$
$validRatio \leftarrow \bar{r} < 1 - \epsilon_{\text{FREE}}$
$validInjectionRatio \leftarrow \bar{r}' \in [1 - \epsilon_{\text{FREE}}, 1 - \epsilon_{\text{IB}}]$
$validDelay \leftarrow t > \tilde{\tau} + \tau_{\text{IB}}$
$validInjectionPeriod \leftarrow t > \tilde{t} + T_{\text{IB}}$
$validUpdate \leftarrow t \leq t' + \tau_{\text{TX}}$
$validAmounts \leftarrow validAmount \wedge validRatio \wedge validInjectionRatio$
$validTimes \leftarrow validDelay \wedge validInjectionPeriod \wedge validUpdate$
if $validAmounts \wedge validTimes$ **then**
    $s_{\text{BANK}} \leftarrow (R - b, S_{\text{D}})$
    $s_{\text{LP}} \leftarrow (B + b, D)$
    $s_{\text{IB}} \leftarrow (t', \tilde{\tau})$
    return TRUE
else
    return FALSE
end if

3.3.4 ExtractDexy

Algorithm 8 ExtractDexy

Require: $s_{\text{O}}, s_{\text{LP}}, s_{\text{BURN}}$, Dexy d, Tx creation time t', Current time t

$(\tilde{t}_{\text{ED}}, \tilde{t}_{\text{RD}}, \tilde{d}, \tilde{\tau}^{\text{ED}}, \tilde{\tau}^{\text{RD}}) \leftarrow s_{\text{BURN}}$
$(B, D) \leftarrow s_{\text{LP}}$
$(P^*_{\text{D}}, F) \leftarrow s_{\text{O}}$
$s'_{\text{LP}} \leftarrow (B, D - d)$ ▷ Removing Dexy from the liquidity pool.
$\bar{r} \leftarrow \bar{r}(P^*_{\text{D}}, s_{\text{LP}})$
$\bar{r}' \leftarrow \bar{r}(P^*_{\text{D}}, s'_{\text{LP}})$
$validRatio \leftarrow \bar{r} \leq 1 - \epsilon^0_{\text{ED}} < 1 - \epsilon_{\text{FREE}}$
$validExtractionRatio \leftarrow \bar{r}' \in \left[1 - \epsilon^1_{\text{ED}}, 1 - \epsilon_{\text{FREE}}\right]$
$validDelay \leftarrow t > \tilde{\tau}^{\text{ED}} + \tau_{\text{ED}}$
$validUpdate \leftarrow t \leq t' + \tau_{\text{TX}}$
$validAmounts \leftarrow validRatio \wedge validExtractionRatio$
$validTimes \leftarrow validDelay \wedge validUpdate$
if $validAmounts \wedge validTimes$ **then**
    $s_{\text{LP}} \leftarrow (B, D - d)$
    $s_{\text{BURN}} \leftarrow (t', \tilde{t}_{\text{RD}}, d, \tilde{\tau}^{\text{ED}}, \tilde{\tau}^{\text{RD}})$
    return TRUE
else
    return FALSE
end if

3.3.5 ReleaseDexy

Algorithm 9 ReleaseDexy

Require: $s_{\text{O}}, s_{\text{LP}}, s_{\text{BURN}}$, Dexy d, Tx creation time t', Current time t

$(\tilde{t}_{\text{ED}}, \tilde{t}_{\text{RD}}, \tilde{d}, \tilde{\tau}^{\text{ED}}, \tilde{\tau}^{\text{RD}}) \leftarrow s_{\text{BURN}}$
$(B, D) \leftarrow s_{\text{LP}}$
$(P^*_{\text{D}}, F) \leftarrow s_{\text{O}}$
$s'_{\text{LP}} \leftarrow (B, D + d)$ ▷ Adding Dexy back into the liquidity pool.
$\bar{r} \leftarrow \bar{r}(P^*_{\text{D}}, s_{\text{LP}})$
$\bar{r}' \leftarrow \bar{r}(P^*_{\text{D}}, s'_{\text{LP}})$
$validRatio \leftarrow \bar{r} \in (1 + \epsilon_{\text{RD}}, 1 + \epsilon_{\text{FREE}})$
$validInjectionRatio \leftarrow \bar{r}' < 1 + \epsilon_{\text{RD}}$
$validDelay \leftarrow t > \tilde{\tau}^{\text{RD}} + \tau_{\text{RD}}$
$validUpdate \leftarrow t \leq t' + \tau_{\text{TX}}$
$validInjectionAmount \leftarrow d \in (0, \tilde{d})$
$validAmounts \leftarrow validRatio \land validInjectionRatio \land validInjectionAmount$
$validTimes \leftarrow validDelay \land validUpdate$
if $validAmounts \land validTimes$ **then**
    $s_{\text{LP}} \leftarrow (B, D + d)$
    $s_{\text{BURN}} \leftarrow (\tilde{t}_{\text{ED}}, t', \tilde{d} - d, \tilde{\tau}^{\text{ED}}, \tilde{\tau}^{\text{RD}})$
    return TRUE
else
    return FALSE
end if

4 Worst Case Scenario and Bank Reserves

Possible failure scenarios for such a design, and how to put restrictions and design rules for the system to ensure stability of the stablecoin token price, will now be considered.

The bank intervenes when there is a sudden drop in the basecoin price provided by the oracle and enough time has passed for the markets to have stabilize themselves, with no interventions, but have failed to do so. In our case, the bank is doing interventions based on the stablecoin price in the liquidity pool in comparison with the oracle price. So, the bank's intervention is about injecting its basecoin reserves into the pool, thereby decreasing the basecoin price to match the oracle price.

Consider an example where the basecoins are ERG, and assume that the oracle price crashes from p to s sharply and stands there, and before the crash there were e of ERG and u of stablecoin in the liquidity pool, with pool's price being p. The worst case then is when no liqudity is put into the pool during the period T. With large enough T and large enough R, this assumption is not very realistic: some traders will buy ERGs with their stablecoins anyway, price is usually failing with swings where traders could mint stablecoins thus increasing ERG bank reserves. However, it would be reasonable to consider worst-case scenario, then in the real world Dexy will be even more durable than in theory.

In this case, the bank must intervene after T units of time, as the price differs significantly, and restore the price in the pool, so set it to s. We denote amounts of basecoins and stablecoins in the pool after the intervention as e' and u', respectively. Then:

- $e * u = e' * u'$.
- The bank must inject E_1 basecoins into the pool, thus after the trade there will be $e' = e + E_1$ basecoins in the pool.
- The new basecoin price in the pool will be $\frac{u'}{e'} = s$, thus $u' = s * (e + E_1)$.
- So, isolating for the amount that must be injected, we obtain $E_1 = \sqrt{\frac{e*u}{s}} - e$ basecoins that must be added into the pool from the bank.

So by injecting E_1 basecoins, the bank recovers the price. However, this is not enough, as now there are O stablecoin units which can be injected into the pool from outside. Again, it is not realistic to assume that all the O stablecoin would be injected, as some of them are simply lost, some would be kept to buy cheap basecoins at the bottom (stablecoins are often used as a bet for a basecoin price decline). However, we need to assume the worst-case scenario again. We also unrealistically assume that all the O tokens are being sold in very small batches that do not significantly affecting the pool price, where after each batch, a seller of a new batch is waiting for a bank intervention to happen (so for T units of time), and sells

only after the intervention. In this case, all the O tokens are being sold at price close to s, so the bank should have about $E_2 = \frac{O}{s}$ basecoins in reserve to buy the tokens back.

Summing E_1 and E_2, we obtain the basecoin reserves the bank should have to be ready for the worst-case scenario: $E_w = E_1 + E_2 = \sqrt{\frac{e*u}{s}} - e + \frac{O}{s}$.

This scenario is the worst-case for the bank (and only the bank) because it is buying all of the external O $stablecoin$ at the worst possible price s, and also sets the new price by burning its own reserves only.

5 Bank and Pool Rules

Based on needed reserves for worst-case scenario estimation, we can consider minting rules accordingly. Similarly to SigUSD [2] (a Djed instantiation on the Ergo blockchain), we can, for example, target for security in case of a 4x price drop, so to consider $R = \frac{p}{s} = 4$. We can also allow minting of stablecoins while there are enough, so not less than E_w, basecoins in reserves. However, in this case, the stablecoin would be non-mintable most of the time, and only during moments of basecoin price going up significantly will it be possible to mint Dexy. As worst-case scenario is based on unrealistic assumptions, unlikely a realistic protocol can be built on top of it.

Thus, we leave worst-case scenario for UI, so dapps implemented to interact with the Dexy protocol may show, for example, collateralization for the worst-case scenario. Having on-chain data analysis, more precise estimations of reserve quality can be made (by considering stablecoins locked in DeFi protocols or stablecoins likely forgotten in wallets).

Instead, we always allow for cautious minting. That is, we allow minting when oracle price is above pool's price (so the bank provides liquidity for arbitrage when the stablecoin price is above the peg), or we allow to mint a little bit (per some time period) when liquidity pool is in good shape. In details, we have two following minting operations, with minting price being the oracle's price p:

- Arbitrage mint: if price reported by the oracle is higher than in the pool, i.e. $p > \frac{u}{e}$, we allow to print enough stablecoin tokens to bring the price to p. That is, the bank allows to mint up to $\delta_u = \sqrt{p * e * u} - u$ stablecoin by accepting up to $\delta_e = \frac{\delta_u}{p}$ basecoins into reserves.

  To instantiate the rule, we can allow for arbitrage minting if the price p is more than $\frac{u}{e}$ by at least 1% for time period T_{arb} (e.g. 1 hour), with the bank charging 0.5% fee for the operation. After arbitrage mint, it is not possible to do another one within 30 minutes, in order to prevent aggressive liquidity minting via chained transactions.

- Free mint: we allow to mint up to $\frac{u}{100}$ stablecoins within time period T_{free}.

To instantiate the rule, we propose to allow for free mint if $0.98 < r < 1.02$*. Minting fee in this case is also* 0.5%*. We propose to set* T_{free} *to* 12 *hours, then bank reserves can grow by* 2% *of LP volume per day when the peg is okay.*

In addition to minting actions, which increase the bank reserves, we define the following action which decreases the bank reserves, but functions as the first stabilization mechanism:

- Intervention: if the price reported by the oracle is lower than the pool price by a significant enough margin, i.e. $r = \frac{p*e}{u}$ is less than or equal to some constant that is set as a protocol parameter, then the bank will provide enough basecoins to restore the pool price to the oracle price.

  To instantiate the rule, we propose to allow the bank to intervene if $r \leq 0.98$ *for time period* $T_{int} = 1\ day$*. During the intervention, the period is reset, so another intervention will only occur after* $T_{int} = 1\ day$ *at least. The bank will provide enough basecoins to get the pool price price to 99.5% of the oracle price max, but making sure to not spend more than 1% of its reserves for that.*

We also state following rules for the liquidity pool (which, otherwise, acts as an ordinary CP-AMM liquidity pool):

- Stopping withdrawals: if r is below some threshold, withdrawals from liquidity providers are stopped (i.e. liquidity providers cannot redeem their LP tokens for basecoins/stablecoins), so only swaps between basecoins and stablecoins are possible.

  To instantiate the rule, we propose to stop withdrawals immediately if $r \leq 0.98$*.*

- Second stabilization mechanism: what if the bank is out of reserves, but the basecoin price in the liquidity pool is still above the oracle basecoin price? In this case we restore price in the pool by removing liquidity, and there are two possible options here:

  1. Burn: if the bank if empty, and r is below some threshold, it is allowed to burn stablecoins in the pool.

     To instantiate the rule, we propose to burn stablecoin if $r \leq 0.95$ *for a time period* T_{burn}*.* T_{burn} *must be quite big, e.g.* $1\ week$*. We burn enough stablecoins to return to the state of stopped withdrawals, so to* $r = 0.98$*. After burning, the period is reset, so another burn will be done only after* T_{burn} *amount of time at least.*

  2. Extract for the future: if the bank is empty and r is below some threshold, it is allowed to extract stablecoins from the pool and lock them in a contract, only to

be released in the future when the stablecoin price is above the stablecoin oracle price (i.e. the inverse of the basecoin price peg).

To instantiate the rule, we propose to extract stablecoins if $r \leq 0.95$ *for time period* T_{burn} *(so the same as in burn). To prioritize extracted funds over arbitrage mint, we do not have delay in releasing contract. We burn enough to return to the state of stopped withdrawals, so close to* $r = 0.98$ *(exactly,* $0.97 \leq r \leq 0.98$*). After extraction, the period is reset, so another extraction will be done after* T_{burn} *amount of time at least.*

In addition, we introduce a large enough redemption fee for liquidity providers to avoid excessive liquidity hopping, e.g. 2%.

6 Stability

With second stabilization mechanism (burning or extraction) in place, the price in the reference market will eventually stabilize. However, for liquidity holders, burning is painful, extraction not as much, but still not desirable. Thus, the Dexy protocol is trying to avoid it (unlike other protocols, such as Gyroscope or Fei, where redemption rate fails and falls below 1 immediately when the collateralization ratio falls under 100%) by giving markets time to self-stabilize, and then doing interventions. However, this could mean slower stabilization, in comparison with other protocols, but slower stabilization helps the bank to play with possible adversaries in an environment with information assymetry, since humans always have access to more information than the algorithmic bank, and with more flexible decision making as well.

Dexy is also cautious about minting new stablecoins, which could mean slow growth of the number of stablecoins issued. This could be inconvenient, especially for big players, but the protocol is focused on stability in the first place.

Please note that the liquidity pool is disincentivizing massive bank runs due to its constant-product nature. Actually, massive bank runs are simpler for the bank to resolve, in comparison to the slow drain of the worst-case scenario.

7 Related Works

Both the Djed [12] and Gluon [8] protocols are considered as crypto-backed algorithmic stablecoins. This is due to the fact that in each one, the entire bank reserve it tokenized, using two tokens: stablecoins and reservecoins. Stablecoins represents the liabilities of the bank, while reservecoins represent the reserve surplus, i.e. equity, after the liabilities are accounted for.

With this perspective in mind, we can view the Dexy protocol as a variant of the Djed and Gluon protocols. The Dexy stablecoins minted by the bank represent its liabilities. Instead of having reservecoins used to tokenize the bank equity, which can be minted by users, we can consider the bank as having one reservecoin all to itself. This reflects the fact that users cannot redeem their stablecoins from the bank and that the bank is liable to know one, wherein reserves can be less than liabilities during bank interventions into the liquidity pool.

8 Implementations

In this section we consider two concrete implementations of the Dexyprotocol that were made for the Ergo blockchain, a gold-pegged stablecoin named DexyGoldand a USD-pegged stablecoin named USE. The oracles used in both stablecoins are implemented using the same underlying oracle protocol.

8.1 Oracle Pools

Oracle security is the most important issue for our stablecoin protocol, since this is the only component that does not function autonomously on-chain. Thus, the protocol needs to reward oracles.

Oracle Pools 2.0 framework [9] has support for paying rewards in custom tokens. For each instance of an oracle, representing a different asset, corresponding ORTs (Oracle Reward Tokens) will be created. These ORTs will be used to reward oracles and developers, e.g. for bounties or audits.

In the dexy protocol, only the bank minting transactions charge an oracle fee in basecoins. These basecoins are sent to a BuyBack contract, which purchases the corresponding ORT available in a LP on a DEX. The purchased ORT tokens are then distributed to the oracle operators. Thus, the oracle operators receive ORT tokens that they can then redeem for the accumulated basecoin fees in the same DEX LP.

8.2 DexyGold

The first variant of the Dexy protocol is pegged to Gold and has been implement on the Ergo blockchain. The current protocol specification outline in Section 3 is modeled after this implementation.

The ORT token for the Gold oracle pool is called GORT. The pool reports the price for 1 kilogram of Gold, however, the stablecoin protocol implementation uses 1 milligram of Gold internally as the peg.

A UI for interacting with the DexyGold protocol is available at `https://cruxfinance.io/dexy/mint`, when selecting the DexyGold option.

8.3 USE

A variant of the Dexy protocol pegged to USD has also been implemented on the Ergo blockchain. It is marketed using the name USE, since there is greater adoption of a USD stablecoin expected over only a gold pegged one, and that this stablecoin is meant to be *used*.

In Section 3, the protocol specification defined α_{IB} as the percentage of the bank's bsecoins reserve that determines the max amount of basecoins to inject into the liquidity pool. USE differs from this specification by assigning α_{IB} as the percentage of the liquidity pool instead of the bank reserve. This ensures that the bank reserves are not depleted as fast, since it is expected that it's basecoin reserves exceeds the basecoin liquidity in the liquidity pool.

A UI for viewing stats of the USEprotocol is available at `https://cruxfinance.io/dexy/analytics`, while interacting with the algorithmic central bank can be done at `https://cruxfinance.io/dexy/mint`, when selecting the USE option.

8.4 Remarks

The ErgoScript smart-contracts follow a separation-of-concerns design pattern, thus each one handles its own part of the protocol instead of having one giant contract, this results in a total of 18 ErgoScript contracts required for the implementation. These 18 contracts are orchestrated together with the corresponding off-chain software for the different protocol interactions and trackers necessary to check the status of the oracle and pool peg, along with the relevant delay periods. The smart-contracts, off-chain code, simulations, and tests can be found in the following repository `https://github.com/kushti/dexy-stable`.

References

[1] ErgoDEX. `https://www.ergodex.io`.

[2] SigUSD. `https://sigmausd.io`.

[3] The Maker Protocol: MakerDAO's Multi-Collateral Dai (MCD) System. [Online], 2017. `https://makerdao.com/en/whitepaper/`.

[4] Ampleforth protocol. [Online], 2018. `https://www.ampleforth.org/`.

[5] XTN - Neutrino Index Token. [Online], 2023. `https://github.com/waves-exchange/neutrino-contract/blob/master/docs/Neutrino%202.0.%20XTN%20Litepaper.pdf`.

[6] Hayden Adams, Noah Zinsmeister, River Keefer, Dan Robinson, and Moody Salem. Uniswap v3 core, 2021. `https://app.uniswap.org/whitepaper-v3.pdf`.

[7] Andrew Breslin. Gyroscope: A Self-Stabilizing, "All-Weather" Reserve-Backed Stablecoin. [Online], 2022. `https://consensys.io/blog/gyroscope-a-self-stabilizing-all-weather-reserve-backed-stablecoin`.

[8] Bruno Woltzenlogel Paleo, Luca D'Angelo, Mohammad Shaheer, and Giselle Reis. Gluon w: A cryptocurrency stabilization protocol. Cryptology ePrint Archive, Paper 2025/1372, 2025.

[9] scalahub. Eip-0023 oracle pool 2.0. https://github.com/ergoplatform/eips/pull/41.

[10] Jiahua Xu, Krzysztof Paruch, Simon Cousaert, and Yebo Feng. Sok: Decentralized exchanges (dex) with automated market maker (amm) protocols. *ACM Computing Surveys*, 2021.

[11] Joachim Zahnentferner, Dmytro Kaidalov, Jean-Frédéric Etienne, and Javier Díaz. Djed: A formally verified crypto-backed pegged algorithmic stablecoin. Cryptology ePrint Archive, Report 2021/1069, 2021. https://eprint.iacr.org/2021/1069.

[12] Joachim Zahnentferner, Dmytro Kaidalov, Jean-Frédéric Etienne, and Javier Díaz. Djed: A formally verified crypto-backed pegged autonomous stablecoin. In *IEEE International Conference on Blockchain and Cryptocurrency*, 2023. https://icbc2023.ieee-icbc.org/program/icbc-2023-final-program.

Received \jreceived